

# COL1000: Lab 10

Semester 2, 2025–26

**Instructions** Upload your code to Gradescope as: <entryNumber>\_q<quesNumber>.py. After uploading, run the autograder to check that all tests pass. Make sure not to include any custom prompt text in `input()` calls, and any extra text in `print()` statements. You must not use the `math`, `ast`, `intertools`, `sympy` libraries.

**Question 1: Merge Sort** Write a program that reads a collection of positive integers from a single line of input (space-separated), sorts them in ascending order by implementing the merge sort algorithm, and prints each sorted element on a separate line. (You must not use in-built sort function for this question).

**Question 2: Matrix Operations** Complete the starter code below to implement a `matrix` class supporting addition, multiplication, and comparison. If an addition or multiplication operation is not possible, your function must print 'Invalid' and return a  $1 \times 1$  matrix containing 0.

```
class matrix:
    def __init__(self, L):
        self.n = len(L)
        self.m = len(L[0])
        self.B = []
        for row in L:
            self.B.append(row.copy())

    def __add__(self, other):
        pass

    def __mul__(self, other):
        pass

    def __eq__(self, other):
        pass

    def display(self):
        for i in range(self.n):
            for j in range(self.m):
                if (j<self.m-1): print(self.B[i][j], end= )
                else: print(self.B[i][j])

def read_and_return_nested_list(r, c):
    L = []
    for i in range(r):
        pass
    return L
```

```

m1 = int(input())
n1 = int(input())
M1 = matrix(read_and_return_nested_list(m1,n1))

m2 = int(input())
n2 = int(input())
M2 = matrix(read_and_return_nested_list(m2,n2))

X = M1+M2
Y = M1*M2

X.display()
Y.display()
print(X==Y)

```

### Example:

Input:

```

2
3
10 20 30
40 50 60
3
2
11 22
33 44
55 66

```

Output:

```

Invalid
0
2420 3080
5390 7040
False

```

**Question 3: Banking System Management** Complete the starter code below to implement a **Bank** class. The **Bank** should manage user accounts internally using a dictionary where the key is the owner's name and the value is their current balance. You must also maintain a counter for the total number of accounts created.

Implement the following methods:

- **open\_account(owner, balance):** Creates a new account in the dictionary and increments the user count. You may assume that **open\_account** is called exactly once for any given owner.
- **deposit(owner, amount):** Adds money to the specified owner's balance if the amount is greater than 0. You must perform this action only if the owner exists.

- `withdraw(owner, amount)`: Deducts money. Print 'Invalid withdrawal' if funds are insufficient, the amount is 0 or less. You must perform this action only if the owner exists.
- `get_balance(owner)`: Returns the current balance of the account (or 0 if they do not exist). You must perform this action only if the owner exists.
- `get_users_count()`: Returns the total number of accounts in the bank.

```
class Bank:
    def __init__(self):
        self.accounts = {}
        self.users_count = 0

    def open_account(self, owner, initial_balance):
        pass

    def deposit(self, owner, amount):
        pass

    def withdraw(self, owner, amount):
        pass

    def get_balance(self, owner):
        pass

    def get_users_count(self):
        pass

bank = Bank()
k = int(input()) # Number of queries

for i in range(k):
    query = input().split()
    cmd = query[0]

    if cmd == OPEN_ACCOUNT:
        bank.open_account(query[1], int(query[2]))
    elif cmd == DEPOSIT:
        bank.deposit(query[1], int(query[2]))
    elif cmd == WITHDRAW:
        bank.withdraw(query[1], int(query[2]))
    elif cmd == GET_BALANCE:
        print(bank.get_balance(query[1]))
    elif cmd == USERS_COUNT:
        print(bank.get_users_count())
```

### Example:

Input:

```
8
OPEN_ACCOUNT Alice 1000
OPEN_ACCOUNT Bob 500
DEPOSIT Alice 200
WITHDRAW Bob 600
WITHDRAW Bob 100
GET_BALANCE Alice
GET_BALANCE Bob
USERS_COUNT
```

Output:

```
Invalid withdrawal
1200
400
2
```

**Question 4: E-Commerce Inventory and Review System** Complete the starter code below to implement an e-commerce platform. You must complete the methods in the `Product` and `Store` classes to handle inventory management, customer purchases, and user reviews.

The system must follow these rules:

- **Purchase:** If a customer attempts to purchase more units than available in stock, the operation should fail and print `Insufficient stock`. Otherwise, reduce the stock accordingly.
- **Reviews:** A user can review a product with a rating between 1 and 5. If the same user submits multiple reviews for the same product, only the latest rating is retained. If a product has no reviews, its average rating is 0.0.
- **Display:** The `display_inventory` method must print products sorted in **descending order of average rating**. If two products have the same rating, sort them in **ascending order of price**.

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock
        self.reviews = {} # username -> rating

    def get_avg_rating(self):
        pass
```

```

class Store:
    def __init__(self):
        self.catalog = {} # name -> Product

    def add_product(self, name, price, stock):
        self.catalog[name] = Product(name, price, stock)

    def restock(self, name, quantity):
        pass

    def purchase(self, name, quantity):
        pass

    def add_review(self, user, name, rating):
        pass

    def display_inventory(self):
        pass
        # Format: {name} {stock} {avg-rating}

amazon = Store()

n = int(input())
for i in range(n):
    name, price, stock = input().split()
    amazon.add_product(name, int(price), int(stock))

k = int(input())
for i in range(k):
    query = input().split()
    if query[0] == PURCHASE:
        amazon.purchase(query[1], int(query[2]))
    elif query[0] == RESTOCK:
        amazon.restock(query[1], int(query[2]))
    elif query[0] == REVIEW:
        amazon.add_review(query[1], query[2], int(query[3]))

print(--- Final Inventory ---)
amazon.display_inventory()

```

### Example:

Input:

3

Laptop 30000 5

Phone 15000 2

Tablet 20000 10

8

PURCHASE Phone 1  
PURCHASE Phone 2  
REVIEW Alice Laptop 5  
REVIEW Alice Phone 5  
REVIEW Bob Laptop 4  
REVIEW Alice Phone 3  
RESTOCK Phone 5  
PURCHASE Phone 2

Output:

Insufficient stock  
--- Final Inventory ---  
Laptop 5 4.5  
Phone 4 3.0  
Tablet 10 0.0